

Project23—ViseFellowship

Repository map

Generated reference (maintain manually or via script)

Synced: 2026-05-14

Abstract

This document summarizes the layout of the `Project23---ViseFellowship` repository with emphasis on the `trade-reconciliation/` package: parsers, reconciliation logic, optional Rust acceleration, tests, sample data, and generated outputs.

1 Repository root

The repository root contains a short pointer `README.md` and convenience copies of CSV inputs at the top level (`trades.csv`, `lots.csv`). All runnable code and canonical data for the exercise live under `trade-reconciliation/`.

2 Directory tree (`trade-reconciliation/`)

```
trade-reconciliation/
  README.md
  requirements.txt
  .gitignore
  data/
    raw/
      trades.csv
      lots.csv
      (FIDELITY-20250729.txt when present)
    processed/
  docs/
    solution_journal.md
    repository_map.tex    <-- this file
  outputs/
    reconciliation_report.csv      (after run)
    reconciliation_summary.json   (after run)
    sample_console_output.txt     (after run)
```

```

rust-core/
  Cargo.toml
  src/main.rs
src/
  __init__.py
  main.py
  rust_bridge.py
  parser.py
  models.py
  reconcile.py
  aggregation.py
  utils.py
tests/
  __init__.py
  test_parser.py
  test_reconcile.py
  test_aggregation.py

```

3 Specific folders

trade-reconciliation/src/ Python package: entry point, parsing, domain models, reconciliation and summaries, CSV/JSON writers, and subprocess bridge to the Rust core.

trade-reconciliation/tests/ pytest suite for parser, reconcile, and aggregation behavior.

trade-reconciliation/data/raw/ Input artifacts: Vise `trades.csv`, `lots.csv`, and (when supplied) fixed-width Fidelity `FIDELITY-20250729.txt`.

trade-reconciliation/data/processed/ Reserved for derived or intermediate data (may be empty).

trade-reconciliation/rust-core/ Optional Cargo project: faster parse/reconcile path invoked from Python when `cargo` is available.

trade-reconciliation/outputs/ Written on each successful run: discrepancy report, JSON summary, and captured console text.

trade-reconciliation/docs/ Narrative notes (`solution_journal.md`) and this repository map.

4 Specific files under src/

File	Role
<code>main.py</code>	Orchestrates the pipeline: tries Rust via <code>rust_bridge.py</code> ; parses Vise lots/trades in Python; validates sells vs. lots; writes <code>outputs/</code> CSV/JSON and console capture.
<code>rust_bridge.py</code>	If <code>cargo</code> and <code>rust-core/Cargo.toml</code> exist, runs <code>cargo run -release</code> with Fidelity and trades paths and reads JSON from a temp file; returns <code>None</code> on failure so Python path runs.
<code>parser.py</code>	Fixed-width Fidelity line parsing (record type D, trade flag, symbol/side/quantity/notional slices) and CSV parsers for Vise trades and lots into <code>models.py</code> types.
<code>models.py</code>	Frozen dataclasses: <code>NormalizedTrade</code> , <code>LotRecord</code> , <code>Discrepancy</code> ; serialization helpers for CSV/JSON-friendly dicts.
<code>reconcile.py</code>	Aggregates trades by (symbol, side); emits discrepancy rows (missing/extra Fidelity, quantity vs. notional mismatch); <code>validate_sell_quantities_against_lots</code> .
<code>aggregation.py</code>	Builds discrepancy count rollups and notional summaries (by side and by ticker) for the JSON summary when using the Python engine.
<code>utils.py</code>	Implied-decimal parsing, money quantization, <code>write_csv</code> / <code>write_json</code> .
<code>__init__.py</code>	Package marker.

5 Rust core

`rust-core/Cargo.toml` Crate manifest for the optional native path.

`rust-core/src/main.rs` CLI: reads Fidelity and Vise trade paths, writes reconciliation JSON for the Python bridge to consume.

6 Tests

File	Focus
<code>tests/test_parser.py</code>	Fidelity/Vise parsing correctness
<code>tests/test_reconcile.py</code>	Aggregation and discrepancy classification
<code>tests/test_aggregation.py</code>	Summary and count helpers

7 Configuration and docs

requirements.txt Python dependencies for the project.

.gitignore Ignores virtualenv, caches, and generated artifacts as configured.

README.md Setup, run (`python -m src.main`), test (`pytest`), and layout overview.

docs/solution_journal.md Working notes and rationale for the assessment.

8 Outputs (generated)

After `python -m src.main` from `trade-reconciliation/`:

outputs/reconciliation_report.csv Row-level discrepancies.

outputs/reconciliation_summary.json Counts, notional summaries, engine (`rust` vs. `python`), lot warnings, timestamp.

outputs/sample_console_output.txt Captured stdout from the last run.